

CORONA: A Coarse-to-Fine Framework for Graph-based Recommendation with Large Language Models

2023036299 김진욱

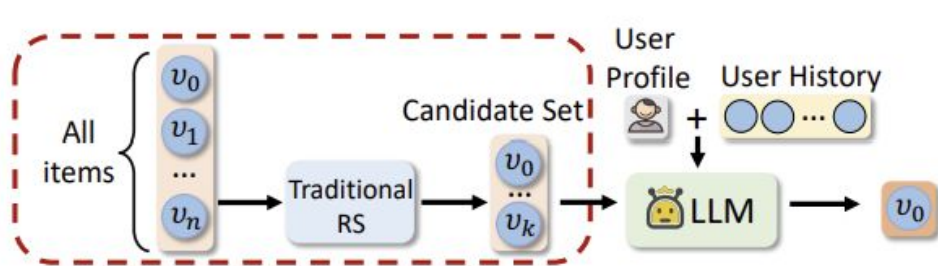
Index

1. Motivation
2. Introduction
3. DataSet
4. Methodology
5. Recommendation performance
6. Conclusion
7. Application

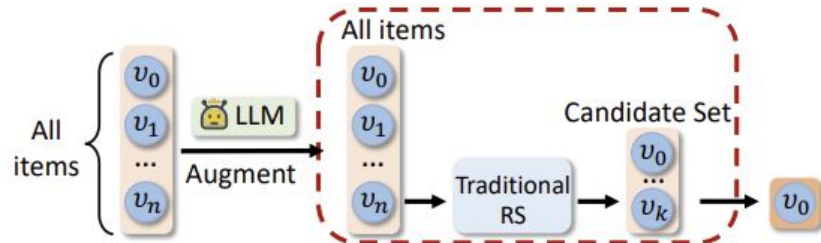
01. Motivation

Motivation

LLM이 여러 도메인에서 주목할만한 성과를 보이면서, GNN을 활용하여 사용자에게 추천 할 아이템을 생성하던 추천 시스템 분야에서도 LLM을 활용한 연구를 시작했다.



Previous methods that apply LLMs after candidate filtering. These methods typically rely on traditional RSs to obtain candidate items, and use LLMs as the final predictors



Previous methods that apply LLMs before candidate filtering. These methods typically use LLMs for data augmentation, and then rely on traditional RSs to obtain candidate items

Motivation

```
You will serve as an assistant to help me summarize which types of users would enjoy a specific business.
I will provide you with the basic information (name, city and category) of that business and also some feedback of users for it.
Here are the instructions:
1. The basic information will be described in JSON format, with the following attributes:
{
  "name": "the name of the business",
  "city": "city where the company is located", (if there is no city, I will set this value to "None")
  "categories": "several tags describing the business" (if there is no categories, I will set this value to "None")
}
2. Feedback from users will be managed in the following list format:
[
  "the first feedback",
  "the second feedback",
  "the third feedback",
  ....
]

2. The information I will give you:
BASIC INFORMATION: a JSON string describing the basic information about the business.
USER FEEDBACK: a List object containing some feedback from users about the business.

Requirements:
1. Please provide your answer in JSON format, following this structure:
{
  "summarization": "A summarization of what types of users would enjoy this business"
  (if you are unable to summarize it, please set this value to "None")
  "reasoning": "briefly explain your reasoning for the summarization"
}
2. Please ensure that the "summarization" is no longer than 200 words.
2. Please ensure that the "reasoning" is no longer than 200 words.
4. Do not provide any other text outside the JSON string.
```

methods that apply LLMs before candidate filtering의 하나인 RLMrec에서 LLM을 활용해 Data Augmentation을 하기 위해 작성한 프롬프트 (출처: github.com/hkuds/rlmrec)

#Question: A user has given high ratings to the following books: <ItemTitleList>. Additionally, we have information about the user's preferences encoded in the feature <UserID>. Using all available information, make a prediction about whether the user would enjoy the book titled <TargetItemTitle> with the feature <TargetItemID>? Answer with "Yes" or "No". \n#Answer:

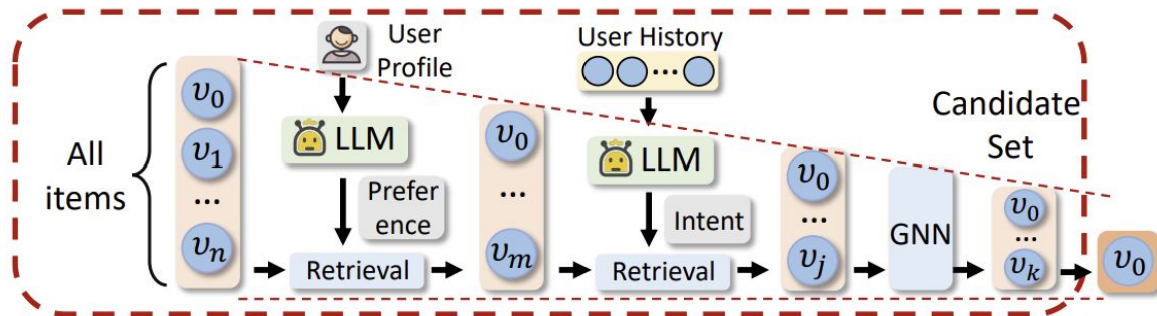
methods that apply LLMs after candidate filtering의 하나인 BinLLM에서 추천 시스템의 결과를 바탕으로 LLM으로 분석하기 위해 작성한 프롬프트 (출처: <https://arxiv.org/abs/2310.15950>)

기존 method는 LLM의 사전학습된 방대한 지식과 추론 능력을 많이 활용하지 못하고, Traditional RS(GNN 기반)의 성능에 크게 의존한다는 문제점이 존재한다.

02. Introduction

Introduction

LLM의 추론 능력을 충분히 활용한 coarse-tofine framework인, Chain Of Retrieval ON grAphs을 제안.



후보 필터링 (**Candidate filtering**) 및 검색에 **LLM**을 직접 배치

- 문맥이 바뀌는 상황에서도 유연하게 대처 가능

03. DataSet

DataSet - Netflix

Datasets	Netflix	MovieLens	Amazon-book
----------	---------	-----------	-------------

:Netflix가 자사 Collaborative Filter 성능 향상을 위해 개최한 Netflix Prize에서 개발자들에게 공개한 데이터

- 사용자 약 48만명, 영화 약 17,700편, 평점(1~5점) 약 1억 50만건으로 구성.
- 85억개($480,000 \times 17,700$)의 User-Item Matrix에서 실제 평점 데이터는 1억 50만개.
- 희소한 평점 데이터만으로 사용자의 미관측 평점을 얼마나 정확하게 예측하는지를 평가하는 척도
- Netflix's Recall : 사용자가 좋아할 만한 영화들을 많이 찾아냈음을 나타냄.
- Netflix's NDCG : 좋아할 만한 영화를 앞쪽 순위에 잘 배치했음을 나타냄.

(User, Movie, Rating, Date)

한계:

명시적 평점(Explicit Feedback, 1~5점)만 포함하며, 영화 도메인에 한정되어 있고 메타데이터가 부족하여 다양한 실제 추천 환경을 완전히 반영하지는 못함

DataSet - MovieLens

Datasets	Netflix	MovieLens	Amazon-book
----------	---------	-----------	-------------

: GroupLens Research에서 연구 목적으로 공개한 사용자들의 영화 평가(Rating) 데이터

- 사용자가 관람한 영화에 부여한 평점을 바탕으로 다른 영화의 내릴 평점을 예측
- ratings.csv : {userId, movieId, rating, timestamp}로 구성. 아직 rating이 존재하지 않는 movieId의 rating을 평가하는 것이 목표
- movies.csv : {movieId, title, genres}로 구성.
- tags.csv : 사용자가 movie에 직접 붙인 태그. ex:) funny, space, romantic
- links.csv : imdb, tmdb 로부터 영화의 정보를 불러와서 추가
- tag Genome : 영화가 1100개 이상의 각각의 태그와 얼마나 관련있는지를 알려줌
- 다양한 크기의 데이터 셋 : Ratings 10만건부터 2500만건까지 다양한 크기의 데이터셋이 존재.

MovieLens는 단순한 사용자-영화 평점 데이터뿐 아니라 장르, 태그, 외부 영화 데이터 연동 정보 및 Tag Genome과 같은 풍부한 메타데이터를 제공한다는 점에서 Netflix Prize 데이터셋과 차별성을 가짐.

하지만 영화 도메인에 한정된 데이터셋이며 명시적 평점 중심으로 구성되어 실제 대규모 추천 서비스의 다양한 사용자 행동을 완전히 반영하지는 못함.

DataSet - Amazon Book

Datasets	Netflix	MovieLens	Amazon-book

:Amazon Review Dataset(Books, Electronics, Beauty, Clothing, Sports, Movies)에서 books 카테고리 데이터셋

- {user_id, asin(도서id), rating, timestamp, review_text, summary}로 구성
- user가 읽는 책의 권수는 매우 한정적이다보니 rating 정보가 매우 sparse
- 베스트셀러 위주로 rating이 존재. (Long-Tail)
- cold - start 리뷰가 매우 많음.
- 책 특성 상 영화 리뷰에 비해 노이즈가 매우 많음.(선물 구매, 충동 구매, 책을 읽지 않고 남긴 잘못된 리뷰 등...)
- 풍부한 메타 데이터 : title, author, publisher, brand, price, category, description, image, sales rank

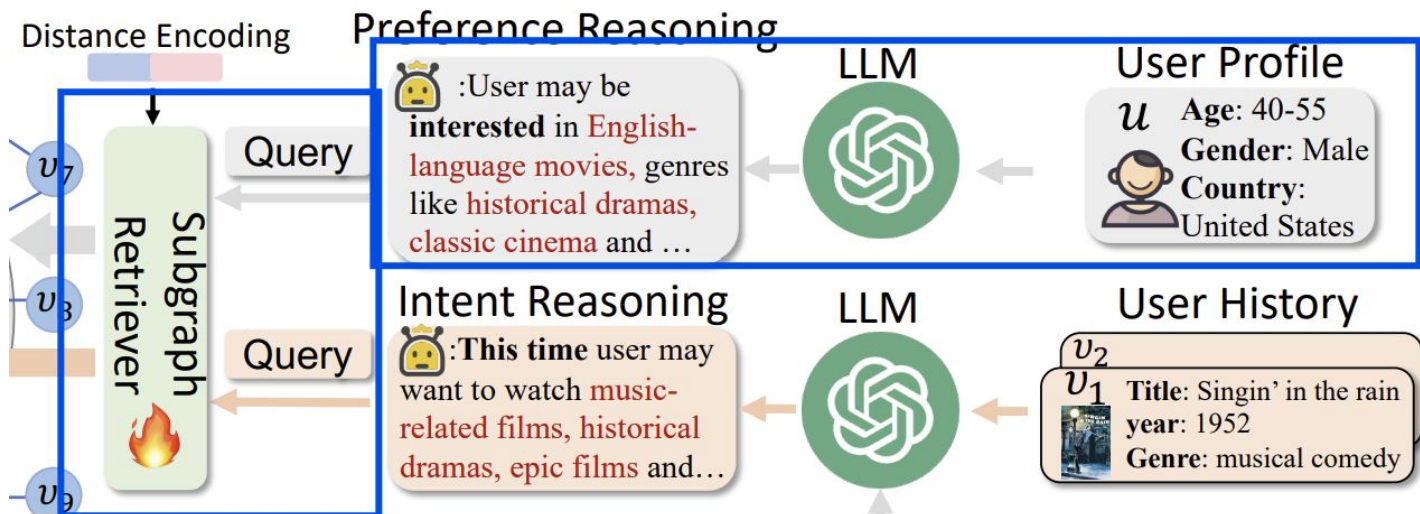
Amazon Books는 Amazon Review Dataset의 하위 데이터셋으로, 풍부한 메타데이터와 대규모 사용자-아이템 상호작용을 제공하며, cold-start와 long-tail 등 실제 추천 환경의 특성을 잘 반영한다.

04. Methodology

Step1. Preference - assisted Retrieval

전체 그래프 G 에서 필요한 하위 그래프만 떼어내는 작업

LLM이 프롬프트를 통해 입력받은 유저의 프로필을 통해 유저의 선호도를 추론한다.
유저의 취향을 임베딩 한 다음, 다른 유저들과 유사도를 측정하여,
취향이 비슷한 유저들을 추려내어 서브 그래프를 구성한다.



Step1. Preference - assisted Retrieval - Implementation (1)

Preference Reasoning :

논문은 profile을 입력으로 받아 선호를 추론하지만,
실제 구현은 user history을 입력받아 profile 자체를 LLM이 생성하면서 선호까지 한 번에 추론한다.

Embedding :

프로파일 텍스트를 SentenceTransformer로 인코딩해,
타깃·기존 유저 구분 없이 모든 유저를 384차원 벡터 한 테이블로 만든다.

```
# =====
# [A] 선호 추론 (Preference Reasoning) - chat_api_query.py
#   유저의 시청 이력을 instruction으로 조합하고, LLM이 인구통계 + 선호를 추론.
# =====
def construct_prompting(item_attribute, item_list):
    # (1) 시청 이력을 "[id] year, title" 형태의 자연어 블록으로 변환
    history_string = "User history:\n"
    for index in item_list:
        row = item_attribute.loc[item_attribute['id'] == index]
        if row.empty:
            continue
        year = row['year'].values[0]
        title = row['title'].values[0]
        history_string += f"[{index}] {year}, {title}\n"

    # (2) LLM이 추론해서 채울 출력 포맷 ('unknown' 금지 -> 반드시 추론)
    output_format = (
        "Please infer the user's information in the format below. "
        "Do not use 'unknown'. You must infer from the patterns given.\n"
        "{ 'summary', 'age', 'gender', 'liked genre', 'disliked genre', "
        "'liked directors', 'country', 'language', 'explanation' }\n"
    )

    # (3) 최종 프롬프트 = 지시문 + 이력 + 출력 포맷
    prompt = ("You are required to generate a user profile based on the interaction "
        | "history below. Use this history to infer the user's characteristics.\n")
    prompt += history_string + output_format
    return prompt
```

```
user_profiles = [
    augmented_user_profiling_dict[index]
    for index in sorted(augmented_user_profiling_dict.keys())
]

## s_model = SentenceTransformer("all-MiniLM-L6-v2")
user_embedding = s_model.encode(user_profiles)
```

Step1. Preference - assisted Retrieval - Implementation(2)

find similar users:

원본 임베딩을 쿼리로, 거리 인코딩을 융합·변환한 다른 유저 임베딩들과 코사인 유사도를 한 번에 계산한다.

torch.topk로 상위 k 유저를 뽑아 최종 유저 집합을 만든다.

Construct SubGraph:

유사한 유저들과 그들이 상호작용한 아이템을 모아,

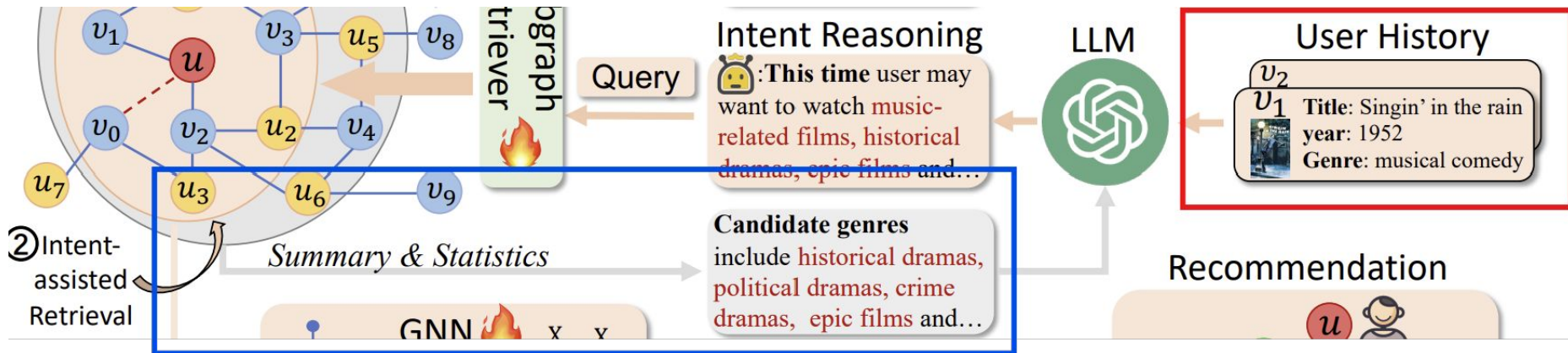
PyTorch Geometric(HeteroData)으로 타깃 유저별 user-item 이분 서브그래프를 구축

```
for index, item in tqdm(retrieved_nodes.items()):
    item = item.cpu()
    retrieved_nodes_np = item.numpy()
    sub_ui_graph = ui_graph[retrieved_nodes_np, :]
    items_connected = sub_ui_graph.nonzero()[1]
    items_connected = np.unique(items_connected)
    sub_ui_graph = sub_ui_graph[:, items_connected]
    sub_iu_graph = sub_ui_graph.T
    ui_graph_dict[index] = sub_ui_graph
    iu_graph_dict[index] = sub_iu_graph
    items_connected_dict[index] = items_connected.tolist()
    total_items_length += len(items_connected)
    row, col = sub_ui_graph.nonzero()
    user_ids = retrieved_nodes_np[row]
    item_ids = col
    data = HeteroData()
    data['user'].num_nodes = n_users
    data['item'].num_nodes = n_items
    edge_index = torch.tensor([user_ids, item_ids], dtype=torch.long)
    data['user', 'interacts', 'item'].edge_index = edge_index
    graphs_dir = f'{data_path}/Graph_RA_Rec/{dataset_name}/graphs'
    os.makedirs(graphs_dir, exist_ok=True)
    torch.save(data, f'{graphs_dir}/{index}.pt')
```

Step2. Intent - assisted Retrieval

1차 Preference 서브 그래프와 User History를 바탕으로 의도를 추론하는 과정

유저의 Preference 서브 그래프 정보와 User History 정보를 통해 Intent를 추론한다.
추출한 의도를 임베딩 한 다음, Preference 서브 그래프 내의 유저들과 유사도를 측정하여
의도가 비슷한 유저들을 추려내어 서브 그래프를 구성한다.



Step2. Intent - assisted Retrieval - Implementation

공식 소스 코드 (<https://github.com/BUPT-GAMMA/CORONA>)에서 **Step2 Implementation**은 존재 X.

Preference(선호도)는 사용자의 포괄적인 취향을 반영하지만, 사용자가 그 아이템을 선택한 의도는 더 세밀하고 단기적일 수 있다. **Intent-assisted Retrieval**은 사용자의 시청 이력을 바탕으로 이 의도를 추론해, 1차 선호 서브그래프 안에서 의도에 더 부합하는 유저·아이템만 2차로 추려 최종 서브그래프 **G'**를 추출하는 과정이다. 이렇게 좁혀진 서브그래프 위에서 다운스트림 **GNN Collaborative Filtering**이 더 정확한 추천을 하도록 돕는다.

Module: Summary & Statistics

Candidate Genres: The candidate items belong to the following genres: *[historical drama, political drama, epic film, music-related film, crime drama...]*

Candidate Categories: The items can be categorized into: *[independent film, documentary, animation ...]*

Years of release: The movies were released in the following periods: *[1930-1950, 1960-1970, ...]*

Module: Intent Reasoning

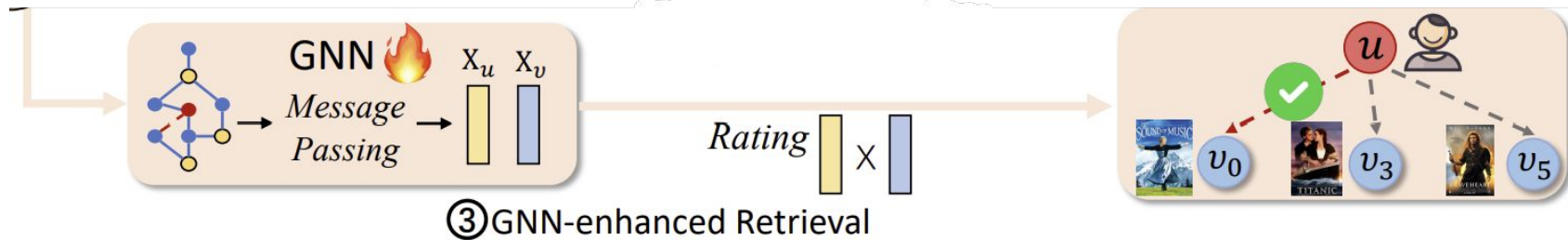
Instruction: Please infer the user's watching intent based on user history and candidate information. The watching intent represents the user's intention for this viewing, and you should make your selection from within the candidate range. Begin by summarizing the relevant information, then provide your intent reasoning predictions using the user history details, candidate sets, and your general knowledge.

User history: No.1: Title: The Last Emperor; Year: 1987; Genre: History drama; No.2: Singin' in the rain...

Candidate summary: Candidate Genres: ...; Candidate Categories: ...; Years of release: ...

Step3. Preference-assisted Retrieval

LLM에 의해 여러번 필터링 된 서브 그래프(Intent에 관한)에 대해서 GNN이 메세지 패싱을 통해 유저와 아이템 간의 상관관계를 계산한 다음, 가장 점수가 높은 Top - N 아이템을 추천낸다.



05. Recommendation performance

Recommendation performance on three datasets

NGCF, LightGCN, GraphPro, GRCN : Graph Based Model

TALLRec, BinLLM : methods that apply LLMs After candidate filtering.

RLMRec, LLMRec : methods that apply LLMs Before candidate filtering

Graph Based CF나 기존의 LLM 결합 CF를 모든 데이터셋에서 CORONA가 능가.

Datasets	Netflix						MovieLens						Amazon-book					
Methods	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50
NGCF	0.0357	0.0163	0.0693	0.0231	0.1090	0.0335	0.1153	0.1007	0.1305	0.1164	0.1636	0.1261	0.0844	0.0637	0.1294	0.0783	0.2151	0.1023
LightGCN	0.0385	0.0152	0.0661	0.0222	0.1252	0.0336	0.0966	0.1013	0.1314	0.1173	0.1647	0.1297	0.0857	0.0646	0.1303	0.0791	0.2154	0.1028
GraphPro	0.0397	0.0174	0.0701	0.0235	0.1263	0.0350	0.1376	0.1121	0.1462	0.1296	0.1757	0.1411	0.0865	0.0656	0.1345	0.0839	0.2203	0.1085
GRCN	0.0392	0.0169	0.0695	0.0260	0.1318	0.0348	0.1175	0.1017	0.1356	0.1192	0.1654	0.1308	0.0862	0.0653	0.1332	0.0826	0.2179	0.1067
TALLRec	0.0498	0.0251	0.0795	0.0338	0.1303	0.0420	0.2521	0.1104	0.3522	0.1748	0.5017	0.1921	0.0857	0.0729	0.1277	0.0838	0.2420	0.1169
BinLLM	0.0502	0.0254	0.0797	0.0340	0.1312	0.0431	0.2584	0.1187	0.3657	0.1771	0.5094	0.1995	<u>0.0972</u>	<u>0.0741</u>	<u>0.1512</u>	<u>0.0939</u>	<u>0.2451</u>	<u>0.1194</u>
RLMRec	0.0504	0.0257	0.0806	0.0341	0.1314	0.0439	<u>0.2595</u>	<u>0.1195</u>	<u>0.3668</u>	<u>0.1779</u>	<u>0.5108</u>	<u>0.2001</u>	0.0968	0.0738	0.1499	0.0913	0.2432	0.1175
LLMRec	<u>0.0526</u>	<u>0.0271</u>	<u>0.0808</u>	<u>0.0342</u>	<u>0.1317</u>	<u>0.0442</u>	0.2582	0.1187	0.3528	0.1750	0.5050	0.1904	0.0862	0.0733	0.1285	0.0840	0.2424	0.1171
CORONA	0.0616	0.0279	0.0938	0.0416	0.1452	0.0487	0.3033	0.1565	0.4214	0.2017	0.5745	0.2507	0.1206	0.0855	0.1857	0.1089	0.3048	0.1299

Performance under item cold-start setting

상호작용이 2회 이하인 **Cold-start** 아이템에 대한 평가 결과, CORONA는 전통적인 **Cold-start** 프레임워크 (DropoutNet, ALDI)뿐만 아니라 **Cold-start** 전용 LLM 추천 모델(LLM-Ins)의 성능을 모두 상회. LLM의 텍스트 추론 능력을 활용하는 것이 **Cold-start** 문제를 효과적으로 극복하고, 양질의 후보 서브 그래프 (Subgraph)를 생성하는 데 크게 기여함을 입증

Datasets	Netflix						MovieLens						Amazon-book					
Methods	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50
DropoutNet	0.0175	0.0083	0.0323	0.0157	0.0414	0.0195	0.1352	0.0538	0.2107	0.1101	0.3622	0.1495	0.0425	0.0264	0.0832	0.0496	0.1482	0.1007
ALDI	0.0247	0.0099	0.0409	0.0161	0.0527	0.0202	0.1436	0.0552	0.2244	0.1102	0.3613	0.1488	0.0525	0.0307	0.0898	0.0517	0.1511	0.1026
TALLRec	0.0124	0.0075	0.0301	0.0152	0.0322	0.0181	0.1224	0.0525	0.2083	0.1099	0.3597	0.1475	0.0459	0.0278	0.0732	0.0472	0.1368	0.1004
BinLLM	0.0218	0.0086	0.0395	0.0158	0.0513	0.0200	0.01373	0.0547	0.2159	0.1095	0.3638	0.1477	0.0514	0.0296	0.0873	0.0508	0.1485	0.1012
RLMRec	0.0243	0.0096	0.0412	0.0164	0.0535	0.0207	0.1576	0.0588	0.2473	0.1105	0.3626	0.1501	0.0571	0.0319	0.0907	0.0526	0.1524	0.1035
LLMRec	0.0119	0.0073	0.0296	0.0142	0.0315	0.0175	0.1163	0.0517	0.2052	0.1095	0.3594	0.1473	0.0454	0.0273	0.0725	0.0464	0.1357	0.0971
LLM-Ins	<u>0.0281</u>	<u>0.0126</u>	<u>0.0490</u>	<u>0.0182</u>	<u>0.0739</u>	<u>0.0195</u>	<u>0.1826</u>	<u>0.0854</u>	<u>0.2689</u>	<u>0.1372</u>	<u>0.4063</u>	<u>0.1665</u>	<u>0.0875</u>	<u>0.0417</u>	<u>0.1167</u>	<u>0.0704</u>	<u>0.1645</u>	<u>0.1206</u>
CORONA	0.0295	0.0130	0.0519	0.0201	0.0791	0.0217	0.2054	0.0933	0.2917	0.1425	0.4582	0.1931	0.0916	0.0453	0.1212	0.0764	0.1788	0.1314

Combinations of different subgraph retrieval methods

동일한 GNN을 사용하더라도 전체 그래프나 단순 거리 기반 추출(Fixed 1/2-hop)을 적용했을 때보다, LLM을 활용하여 SubGraph를 추출한 CORONA 방식을 적용했을 때 추천 성능이 압도적으로 향상

CORONA를 통해 추출한 서브 그래프가 GCN/GT 불문하고 잘 작동 하였으며, 추천에 필요한 내용을 잘 포함하고 있음을 의미.

Datasets	Netflix						MovieLens						Amazon-book					
Variants	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50
GCN (full graph)	0.0307	<u>0.0179</u>	<u>0.0662</u>	<u>0.0217</u>	<u>0.1012</u>	0.0247	0.1056	0.0808	<u>0.1371</u>	0.0993	<u>0.1600</u>	<u>0.1208</u>	<u>0.0786</u>	0.0405	0.1121	0.0556	0.1845	<u>0.0797</u>
GCN (fixed 1-hop)	<u>0.0319</u>	0.0107	0.0617	0.0169	0.0981	0.0217	<u>0.1123</u>	0.0804	0.1320	0.1037	0.1599	0.1181	0.0773	0.0427	0.1133	0.0516	<u>0.1925</u>	0.0790
GCN (fixed 2-hop)	0.0305	0.0097	0.0632	0.0184	0.0987	<u>0.0274</u>	0.1092	<u>0.0912</u>	0.1244	<u>0.1086</u>	0.1495	0.1178	0.0726	<u>0.0443</u>	<u>0.1135</u>	<u>0.0582</u>	0.1882	0.0796
GCN (CORONA)	0.0616	0.0279	0.0938	0.0416	0.1452	0.0487	0.3033	0.1565	0.4214	0.2017	0.5745	0.2507	0.1206	0.0855	0.1857	0.1089	0.3048	0.1299
GT (full graph)	0.0302	0.0104	0.0641	<u>0.0192</u>	0.1008	0.0285	<u>0.1055</u>	0.0907	0.1235	<u>0.1074</u>	0.1482	0.1170	<u>0.0719</u>	0.0435	0.1127	0.0571	0.1854	0.0787
GT (fixed 1-hop)	0.0294	0.0081	0.0655	0.0152	0.0988	<u>0.0342</u>	0.1035	<u>0.1039</u>	0.1170	0.0828	<u>0.1501</u>	0.1137	0.0672	0.0330	0.1109	0.0451	0.1835	0.0787
GT (fixed 2-hop)	<u>0.0345</u>	<u>0.0117</u>	<u>0.0665</u>	0.0160	<u>0.1119</u>	0.0305	0.1044	0.0864	<u>0.1370</u>	0.1043	0.1463	<u>0.1193</u>	0.0696	<u>0.0453</u>	<u>0.1187</u>	<u>0.0601</u>	<u>0.1855</u>	<u>0.0790</u>
GT (CORONA)	0.0628	0.0318	0.0931	0.0294	0.1468	0.0471	0.3072	0.1528	0.4165	0.1988	0.5504	0.2619	0.1203	0.1028	0.1928	0.1202	0.3011	0.1221

Ablation study

Intent Reasoning 모듈 제거(w/o Intent Reasoning) :

- 여타 변형 모델들과 비교하여 대부분의 평가지표에서 가장 저조한 성능을 기록.
- Preference보다 최근 이력을 통한 '단기적 의도(Dynamic Intent)'를 추론하는 것이 추천 성능 향상에 더 결정적인 역할을 하며, Intent Reasoning이 CORONA 프레임워크의 핵심 모듈임을 입증

Datasets	Netflix						MovieLens						Amazon-book					
Methods	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50	R@10	N@10	R@20	N@20	R@50	N@50
w/o Preference-assisted Retrieval	0.0576	0.0227	0.0848	0.0343	0.1295	0.0431	0.2886	0.1375	0.3873	0.1970	0.5306	0.1829	0.1112	0.0747	0.1535	0.0909	0.2228	0.1036
w/o Intent-assisted Retrieval	0.0529	0.0208	0.0785	0.0329	0.1006	0.0418	0.2694	0.1092	0.3590	0.1913	0.5129	0.1768	0.1037	0.0764	0.1643	0.0937	0.2028	0.0961
w/o GNN-enhanced Retrieval	<u>0.0591</u>	<u>0.0264</u>	<u>0.0917</u>	<u>0.0359</u>	<u>0.1304</u>	<u>0.0457</u>	<u>0.2958</u>	<u>0.1434</u>	<u>0.4088</u>	<u>0.1943</u>	<u>0.5531</u>	<u>0.1905</u>	<u>0.1189</u>	<u>0.0814</u>	<u>0.1729</u>	<u>0.0968</u>	<u>0.2311</u>	<u>0.1106</u>
w/o Preference Reasoning	0.0506	0.0187	0.0759	0.0297	0.0954	0.0386	0.2532	0.1158	0.3316	0.1741	0.4819	0.1703	0.0933	0.0653	0.1490	0.0875	0.1986	0.0887
w/o Intent Reasoning	0.0451	0.0142	0.0735	0.0264	0.0933	0.0371	0.2317	0.1173	0.3221	0.1476	0.4609	0.1659	0.0789	0.0621	0.1215	0.0839	0.1982	0.0835
Full CORONA	0.0616	0.0279	0.0938	0.0416	0.1452	0.0487	0.3033	0.1565	0.4214	0.2017	0.5745	0.2507	0.1206	0.0855	0.1857	0.1089	0.3048	0.1299

06. Conclusion

Conclusion

LLM을 전처리(데이터 증강)나 후처리(최종 순위 매기기)에만 쓰는 것이 아니라, 방대한 전체 아이템 풀에서 대략적인 범위(**Coarse-grained**)를 좁혀나가는 검색(**Retrieval**) 과정에 LLM 투입. **Coarse-to-Fine** 프레임워크의 우수성을 성공적으로 입증

**User Profile:** Age: 40-55; Gender: Male; Country: United States; Language: English; Liked directors: Bernardo Bertolucci

**Preference Reasoning:** The user likely enjoys **historical dramas**, **epic films** that reflect deep social and personal themes. Prefers visually rich and thematic films (inspired by Bernardo Bertolucci). Likely favors **English-language movies** from the 1970s-1990s.

**Summary:** Candidate genres include: **historical dramas**, **epic films**, crime dramas, war films, **classic cinema**...

**User History:** Title: The Last Emperor ; Year: 1987; Genre: Historical Drama; Title: The Cook and Other Treasures; Year: 1970; Genre: Classic cinema; Title: Journey: 2001; Year: 2000; Genre: classic cinema; Title: Reds; Year: 1981; Genre: Historical Drama;

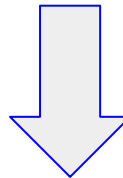
**Intent Reasoning:** The user's viewing preferences lean towards serious, thematic, and genre-specific films, particularly **historical dramas**, **epic films**, music-related films and **international films**.

Final Recommendation @5: Schindler's List (Historical drama, 1993); Crouching Tiger, Hidden Dragon (International film, 2000); Ran (International film, 1985); Lawrence of Arabia (Epic film, 1962); The Godfather (Classic cinema, 1982).

07. Application

Preference - assisted Retrieval Application

- CORONA를 NAVER 대화형 쇼핑 도메인에 적용할 경우, Preference Reasoning 단계에서 사용자의 NAVER Profile을 User Profile로 활용할 수 있다
- User Profile에는 Age, Gender, Country 등의 기본 정보뿐 아니라, NAVER 내 다양한 활동 이력을 반영하여 사용자의 선호를 더욱 정교하게 추론할 수 있을 것이다.



Preference Reasoning

🤖 :User may be interested in **English-language movies**, genres like **historical dramas**, **classic cinema** and ...

LLM

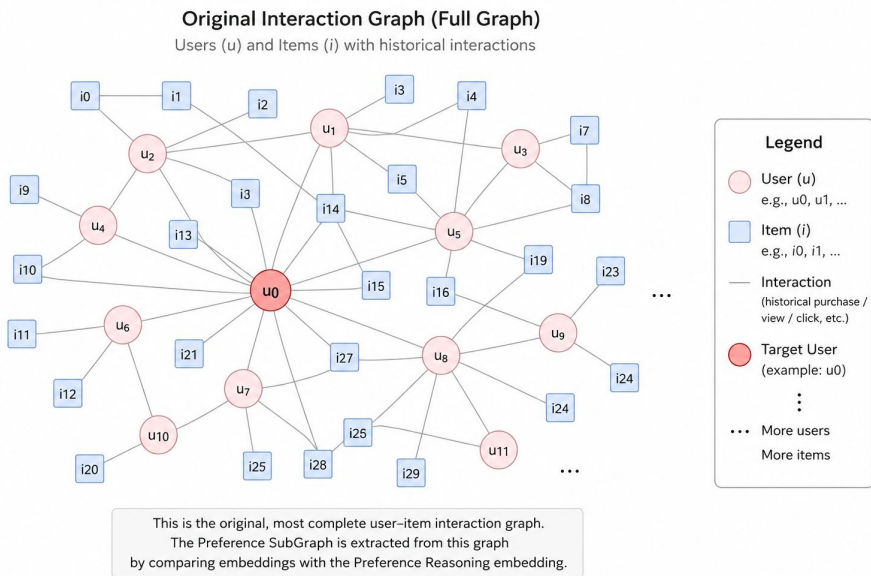


User Profile

u Age: 40-55
Gender: Male
Country: United States

Graph Construction

Preference Reasoning으로 생성된 Query Embedding과 원본 Graph의 User Embedding을 비교하여 Preference SubGraph를 추출한다. 따라서 원본 Interaction Graph 구축이 필요하다.



User Node

Basic Profile

- User ID
- Age
- Gender
- Region
- Membership Period

Behavior History

- Search History
- Recent Viewed Items
- Purchase History
- Favorite Brands
- Preferred Categories
- Preferred Price Range
- Review Keywords
- Shopping Interests

Representation

- Profile Embedding

Item Node

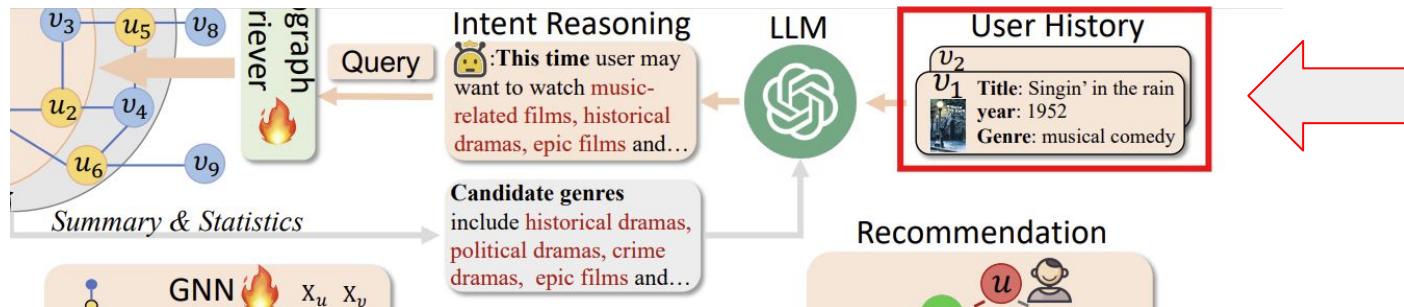
- Item ID
- Product Name
- Category
- Brand
- Price
- Description (Text)
- Image Embedding

User-Item Edge

- Purchase
- Click
- Add to Cart
- Wishlist
- Review
- View
- Interaction Count
- Timestamp

Intent - assisted Retrieval Application

User History에는 사용자의 N스토어 거래 이력뿐만 아니라, 네이버 서비스 내 활동 이력(최근 조회 상품, 찜한 상품, 리뷰 작성 내역, 관심 스토어 등)을 활용할 수 있다. 또한 **Intent Reasoning**의 목적은 사용자의 장기적인 **Preference**와 현재의 구매 **Intent** 사이의 차이를 포착하는 것이므로, 네이버 내 다양한 행동 데이터를 활용하여 사용자의 현재 구매 의도를 더욱 정교하게 추론할 수 있을 것이다.



주문/배송내역

찜한 상품

최근 본 상품

리뷰 작성

작성 가능한 리뷰

내가 작성한 리뷰

관심 스토어

Application Conclusion

CORONA는 LLM을 활용하여 Amazon Books, Netflix, MovieLens와 같은 추천 시스템 벤치마크 데이터셋에서 사용자의 Preference를 넘어 구매 Intent까지 추론하고, Recommendation Performance를 통해 이러한 접근이 추천 성능 향상에 효과적임을 보였다.

다만 실험에 사용된 데이터셋은 주로 평점이나 구매 이력과 같은 사용자-아이템 상호작용 데이터를 기반으로 학습 및 평가가 이루어졌으며, 검색 기록, 최근 조회 상품, 찜, 실시간 대화 맥락과 같은 다양한 행동 데이터는 충분히 활용하기 어렵다는 한계가 있다.

반면 NAVER 대화형 쇼핑 도메인에서는 사용자와 챗봇 간의 지속적인 상호작용뿐만 아니라 검색 기록, 쇼핑 활동, 관심 상품 등 다양한 행동 데이터를 함께 활용할 수 있으므로, 사용자의 현재 구매 Intent를 더욱 정교하게 추론하고 보다 개인화된 추천을 제공할 수 있을 것으로 기대된다.